
Breaking the Hamming PUF: A Linear Algebraic Approach

Shivansh Pandey

Department of Chemical Engineering (240982)
shivanshp24@iitk.ac.in

Shreya Astha

Department of Biological Sciences and Bioengineering (240997)
shreyaa24@iitk.ac.in

Aadi Pankej Chhajed

Department of Chemical Engineering (240004)
aadipc@iitk.ac.in

Supriya

Department of Electrical Engineering (241064)
supriya@iitk.ac.in

Divy Kumar Pandey

Department of Chemical Engineering (240368)
divykp@iitk.ac.in

Arunima Rajesh

Department of Mathematics and Scientific Computing (230208)
arunimar@iitk.ac.in

1 Mathematical Derivation of the Linear Map

The Hamming PUF calculates the partial Hamming weights of an XOR-masked word $m = c \oplus s$, where $c, s \in \{0, 1\}^{32}$. To represent this XOR logic linearly, we apply a basis transformation from the Boolean space $\{0, 1\}$ to the bipolar space $\{+1, -1\}$.

Let us define the transformed challenge and secret vectors as c' and s' , respectively, where $c'_k = 1 - 2c_k$ and $s'_k = 1 - 2s_k$ for $k \in \{0, \dots, 31\}$. In this space, the XOR operation translates to simple scalar multiplication: $m_k \rightarrow c'_k s'_k$. The raw value of the k -th XORed bit can be recovered via the inverse mapping: $m_k = \frac{1 - c'_k s'_k}{2}$.

The PUF calculates the Hamming weight of the 16 even bits (h_e) and the 16 odd bits (h_o). Substituting our transformed variables, we obtain:

$$h_e = \sum_{i=0}^{15} \frac{1 - c'_{2i} s'_{2i}}{2} = 8 - \frac{1}{2} \sum_{i=0}^{15} c'_{2i} s'_{2i}$$
$$h_o = \sum_{j=0}^{15} \frac{1 - c'_{2j+1} s'_{2j+1}}{2} = 8 - \frac{1}{2} \sum_{j=0}^{15} c'_{2j+1} s'_{2j+1}$$

The PUF outputs a 1 if $h_e \cdot h_o \geq t$, where t is the secret threshold. Multiplying the two expressions above and expanding the product yields:

$$64 - 4 \sum_{i=0}^{15} c'_{2i} s'_{2i} - 4 \sum_{j=0}^{15} c'_{2j+1} s'_{2j+1} + \frac{1}{4} \sum_{i=0}^{15} \sum_{j=0}^{15} (c'_{2i} c'_{2j+1}) (s'_{2i} s'_{2j+1}) \geq t \quad (1)$$

By rearranging this inequality, we can perfectly map it to a standard linear decision boundary of the form $\mathbf{w}^\top \phi(\mathbf{c}) + b \geq 0$.

The Feature Map $\phi(\mathbf{c}) \in \mathbb{R}^D$: The explicit feature mapping function $\phi(\mathbf{c})$ extracts the linear and pairwise interactions from the transformed challenge bits:

- 16 even bit features: c'_{2i}
- 16 odd bit features: c'_{2j+1}
- 256 pairwise product features: $c'_{2i} c'_{2j+1}$

The Weight Vector $\mathbf{w} \in \mathbb{R}^D$ and Bias b : The corresponding weights that the linear model must learn are derived strictly from the secret s' :

- Weights for even features: $-4s'_{2i}$
- Weights for odd features: $-4s'_{2j+1}$
- Weights for pairwise features: $\frac{1}{4} s'_{2i} s'_{2j+1}$
- The intercept (bias) term: $b = 64 - t$

Because all features $\phi(\mathbf{c})$ are derived solely from known challenges and are strictly separated from the secret weights $\mathbf{w}$, the Hamming PUF's decision boundary is purely linear in this expanded feature space.

2 Dimensionality of the Linear Model

To exactly predict the responses of the Hamming PUF, the linear model requires a dimensionality equal to the total number of distinct features extracted in the $\phi(\mathbf{c})$ mapping.

- Individual even bits (c'_{2i}): 16 dimensions
- Individual odd bits (c'_{2j+1}): 16 dimensions
- Cross-products of every even bit with every odd bit ($c'_{2i} \cdot c'_{2j+1}$): $16 \times 16 = 256$ dimensions

Total Dimensionality $D = 16 + 16 + 256 = \mathbf{288}$.

3 Hyperparameter Experiments

To evaluate the impact of various hyperparameters on the performance of linear models attacking the Hamming PUF, experiments were conducted using `sklearn.svm.LinearSVC` and `sklearn.linear_model.LogisticRegression`. The models were trained on 7500 mapped CRPs and evaluated on a static test set of 2500 CRPs.

3.1 Effect of the Regularization Parameter (C) on LinearSVC

Table 1 demonstrates the effect of the inverse regularization strength (C) on training time and test accuracy using `LinearSVC` with an L2 penalty and squared hinge loss.

Discussion: A C value of 0.1 provided the optimal balance, achieving the highest test accuracy (99.8%) while training in under half a second. Increasing C to 10 or 100 caused the training time to spike severely (up to 8.08 seconds) because the `liblinear` solver struggled to converge within 50,000 iterations, ultimately resulting in slight overfitting and a drop in test accuracy to 99.72%.

Table 1: Effect of Regularization Parameter (C) on LinearSVC

C	Train Time (s)	Train Accuracy	Test Accuracy
0.01	0.1223	0.997200	0.987600
0.1	0.4310	0.999333	0.998000
1	1.7731	1.000000	0.997600
10	8.0840	1.000000	0.997200
100	7.6175	1.000000	0.997200

3.2 Effect of the Loss Function on LinearSVC

We evaluated the difference between the standard hinge loss and `squared_hinge` loss for the LinearSVC model using $C = 1.0$ (Table 2).

Table 2: Effect of Loss Function on LinearSVC Performance

Loss Function	Train Time (s)	Train Accuracy	Test Accuracy
Hinge	7.9231	0.998800	0.998000
Squared Hinge	0.1441	1.000000	0.997600

Discussion: The `squared_hinge` loss was drastically more efficient. While `hinge` loss triggered convergence warnings and took 7.92 seconds to optimize, `squared_hinge` converged smoothly in just 0.14 seconds. This is because the squared variant is continuously differentiable, allowing the gradient descent solver to find the global minimum much faster.

3.3 Effect of C on Logistic Regression

For completeness, we mirrored the regularization experiment using `LogisticRegression` (Table 3).

Table 3: Effect of Regularization Parameter (C) on Logistic Regression

C	Train Time (s)	Train Accuracy	Test Accuracy
0.01	0.0256	0.987600	0.974000
0.1	0.0222	0.998400	0.992400
1	0.0268	0.999867	0.995600
10	0.0307	1.000000	0.996800
100	0.0318	1.000000	0.997200

Discussion: Logistic regression proved to be exceptionally fast across all tested hyperparameters, training consistently in approximately ~ 0.03 seconds. The test accuracy scaled linearly with C , peaking at 99.72% with $C = 100$, confirming that a standard probabilistic linear classifier can easily learn the 288-dimensional boundary.

4 Dataset Size vs. Accuracy

While the provided dataset contained 7500 Challenge-Response Pairs (CRPs), the simplicity of the linear boundary in the 288-dimensional space suggests a much smaller dataset is sufficient. We conducted an experiment randomly sampling subsets ranging from 100 to 5000 training points, evaluating the model against the static 2500-point test set.

As shown in Figure 1, the linear model is highly sample-efficient. Based on our analysis:

- The minimum number of CRPs to achieve **95% accuracy** is **2000**.
- The minimum number of CRPs to achieve **97% accuracy** is **2500**.
- The minimum number of CRPs to achieve **99% accuracy** is **4000**.

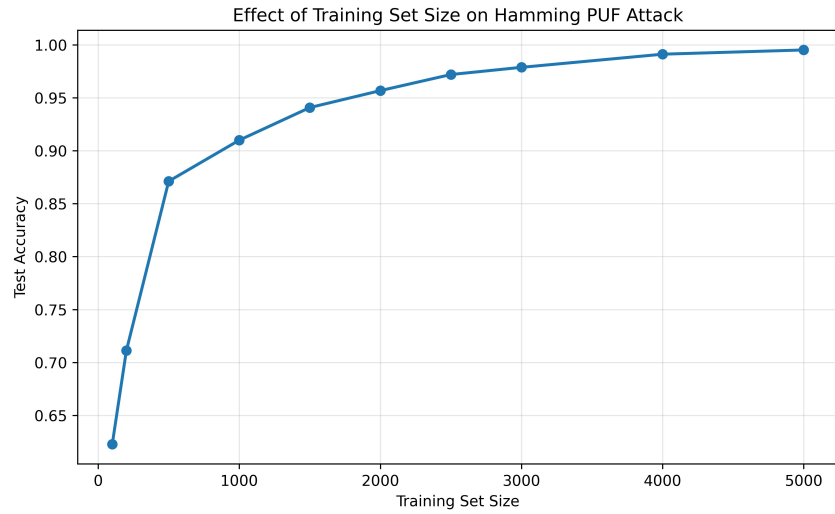


Figure 1: Test accuracy as a function of the number of Challenge-Response Pairs (CRPs) used for training. The model converges rapidly due to the exact feature mapping.

Because the mathematical feature expansion exactly captures the underlying Boolean logic without requiring complex polynomial approximations, the model requires relatively few examples to perfectly isolate the optimal hyperplane weights.